Elective Course
Course Code: CS4103
Autumn 2026-27

Lecture #13

Artificial Intelligence for Data Science

Module-2: PROBLEM SOLVING BY SEARCH

Adversarial Search Problem--- Games [Part-II]

Course Instructor:

Dr. Monidipa Das
Assistant Professor
Department of Computational and Data Sciences
Indian Institute of Science Education and Research Kolkata, India 741246

Home Assignment

Consider the input files (*coordinates.txt*, and *graph_data.csv*) containing the details of coordinates of the cities and the road connectivity among them, respectively.

Write a menu-driven Python code that gives users options, such as **Greedy Best First Search**, **UCS**, and **A* search**, to ***find a route as well as cost*** for reaching *any destination city* from *a given source city*. *Both the cities should be taken as inputs from the user.*

The program should also show the visual representation of the city road network along with indication of source (in text), destination (in text), and the highlighted path as suggested by the selected algorithm.

Node	X	Y
C1	1	9
C2	3	7
C3	2	4
C4	1	1
C5	6	8
C6	6	5
C7	5	3
C8	5	1
C9	9	3
C10	9	7

coordinates.txt

Note:
- Regarding visualization, you may take help from Slide#19 and Slide#23 of Lecture#09

Source	Destination	Cost
C1	C2	4
C2	C1	4
C2	C3	4
C2	C5	4
C3	C2	4
C3	C6	5
C4	C7	11
C4	C8	5
C5	C2	4
C5	C6	9
C6	C3	5
C6	C5	9
C6	C7	8
C6	C9	5
C7	C4	11
C7	C6	8
C7	C8	2
C8	C4	5
C8	C7	2
C8	C9	7
C9	C6	5
C9	C8	7
C9	C10	4
C10	C9	4

graph_data.csv

To be done by a group of ***maximum four (04)*** students;
Submission:
Submit by **30-AUG-2026** (through email with subject "CS4103: Assignment on Road Network" and cc to your group-mates, if there is any)
Please write the code yourself. DO NOT use AI tools to develop the code.

Desired Output

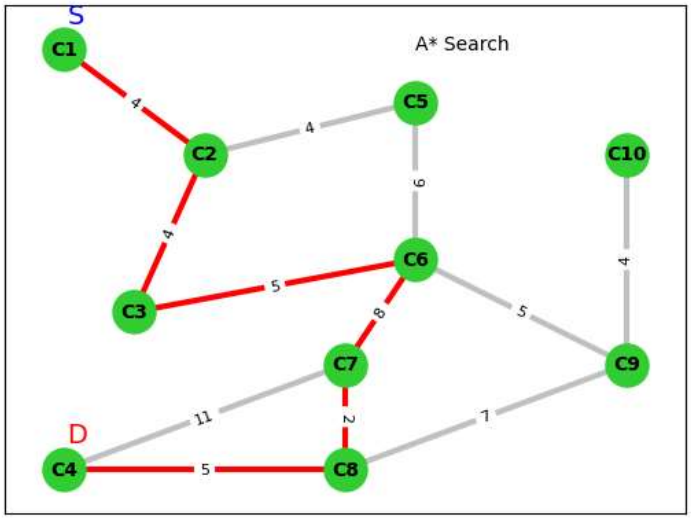
```
==================
OPTIONS AVAILABLE:
==================
1. Uniform Cost Search (UCS)
2. Greedy Best First Search
3. A* Search
4. Exit

Please select an option from above: 3
See the road network and enter the source city:C1
See the road network and enter the destination city:C4
You have selected A* Search. Following is the suggested path:

Path: C1-->C2-->C3-->C6-->C7-->C8-->C4
Path Cost: 28

==================
OPTIONS AVAILABLE:
==================
1. Uniform Cost Search (UCS)
2. Greedy Best First Search
3. A* Search
4. Exit

Please select an option from above: 4
Thank you!....Good bye.....
```



Games as Adversarial Search

- **States:** board configurations
- **S0 i.e. Initial state:** The board position and which player will move
- **PLAYER(s):** Defines which player has the move in a state
- **ACTIONS(s):** Returns the set of legal moves in a state.
- **RESULT(s, a) i.e. Transition model/Successor function:** defines the result of a move; returns list of (state, action) pairs, each indicating a legal move and the resulting state
- **TERMINAL-TEST(s):**
  - determines when the game is over
- **UTILITY(s,p) i.e. Utility function:**
  - gives a numeric value in terminal states *s* for a player *p*

  (e.g., -1, 0, +1 for loss, tie, win)

Evaluation function

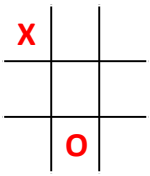


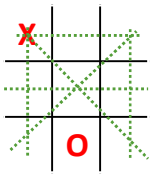
- **Evaluation function** or **static evaluator** is used to evaluate the "goodness" of a game position.
  - Contrast with heuristic search where the evaluation function was a non-negative estimate of the cost from the start node to a goal and passing through the given node
- The **zero-sum assumption** allows us to use a single evaluation function to describe the goodness of a board with respect to both players.
  - $f(n) >> 0$: position n good for player A and bad for player B
  - $f(n) << 0$: position n bad for player A and good for player B
  - $f(n)$ $near$ 0: position n is a neutral position
  - $f(n) = +1$: win for player A
  - $f(n) = -1$: win for player B

Evaluation function examples

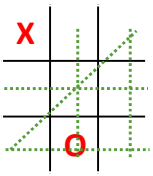


- Example of an evaluation function for **Tic-Tac-Toe**:
  - $f(n) = [\#of\ win\ path\ for\ player\ A] - [\#of\ win\ path\ for\ player\ B]$
    where a win path is a complete row, column, or diagonal





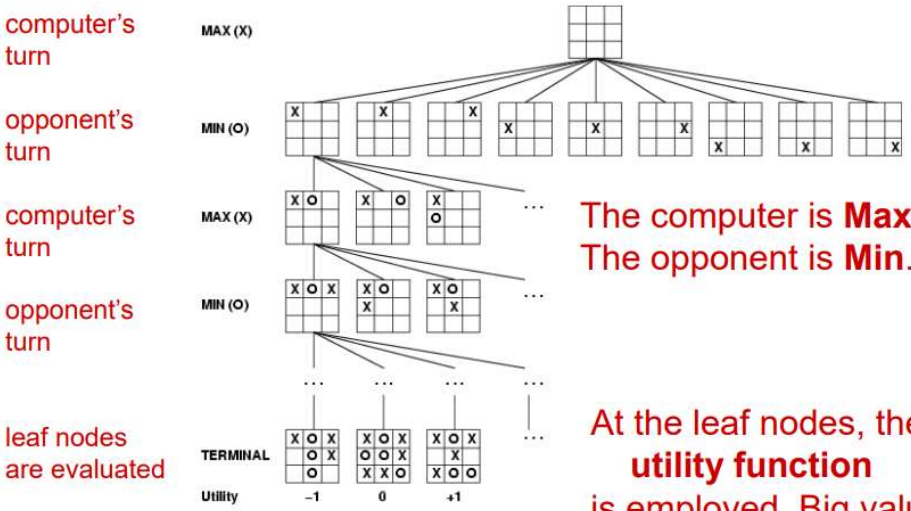
- Alan Turing's function for **chess:**
  - $f(n) = w(n)/b(n)$
    where $w(n)$ = sum of the point value of white's pieces and $b(n)$ = sum of black's



- Most evaluation functions are specified as a weighted sum of position features:
  - $f(n) = w_1 * feature1(n) + w_2 * feature2(n) + \ldots + w_k * featurek(n)$

  Example features for chess are piece count, piece placement, squares controlled, etc.

Game Tree (2-player, Deterministic, Turns)



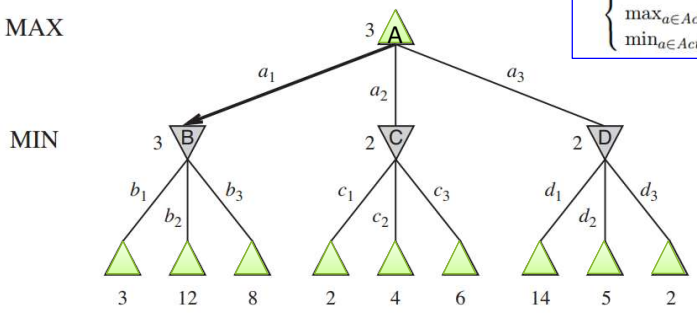
The computer is **Max**.
The opponent is **Min**.

At the leaf nodes, the **utility function** is employed. Big value means good, small is bad.

Minimax

- Perfect play for deterministic games
- **Idea:** choose move to position with highest minimax value
  = best achievable payoff against best play
- E.g., 2-ply game:

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if TERMINAL-TEST}(s) \\ \max_{a \in Actions(s)} \text{MINIMAX}(\text{RESULT}(s,a)) & \text{if PLAYER}(s) = \text{MAX} \\ \min_{a \in Actions(s)} \text{MINIMAX}(\text{RESULT}(s,a)) & \text{if PLAYER}(s) = \text{MIN} \end{cases}$$



MiniMax Terminology



- **Move:** a move considering both players
- **Ply:** a half-move
- **Utility function:** the function applied to **leaf nodes**
- **Backed-up value**
  - of a max-position: the value of its largest successor
  - of a min-position: the value of its smallest successor
- **Minimax procedure:** search down several levels; at the bottom level apply the utility function, back-up values all the way up to the root node, and that node selects the move.

Minimax algorithm Adversarial analogue of DFS

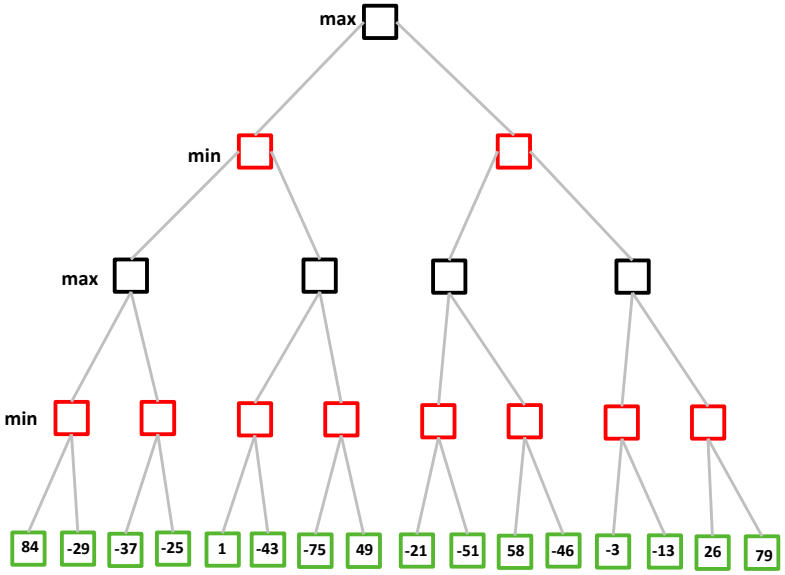



```
function MINIMAX-DECISION(state) returns an action
  return arg max_{a ∈ ACTIONS(s)} MIN-VALUE(RESULT(state, a))

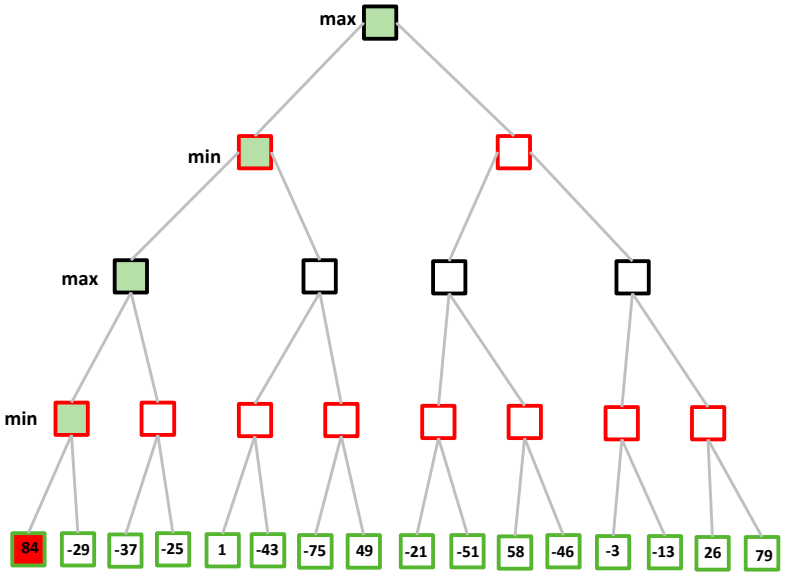
function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s, a)))
  return v

function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← ∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s, a)))
  return v
```

Example

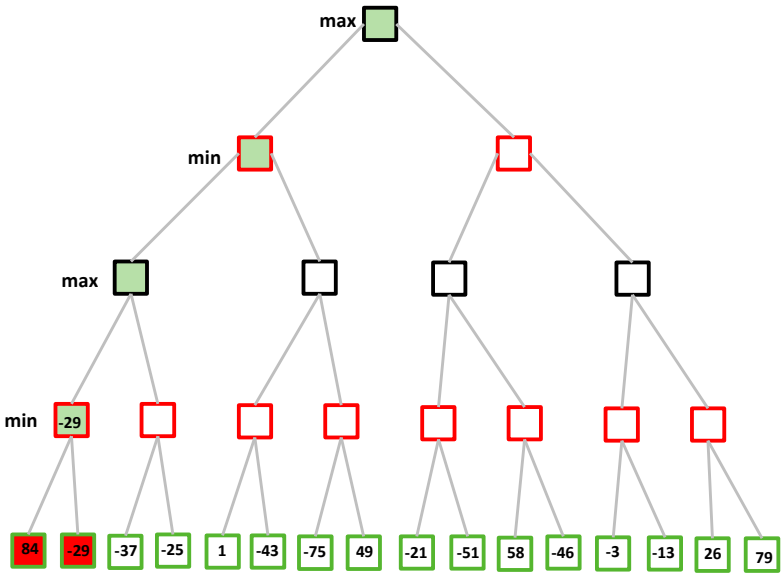


Example



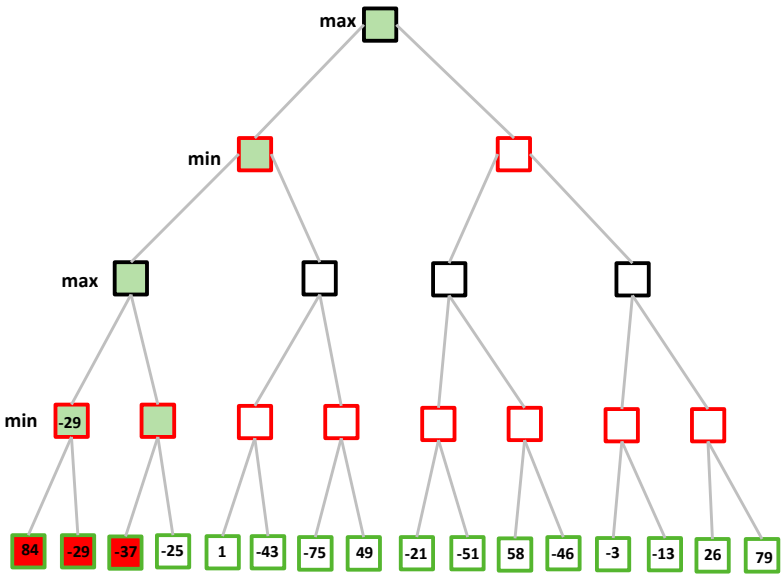
Example



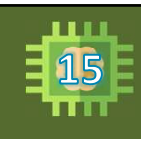


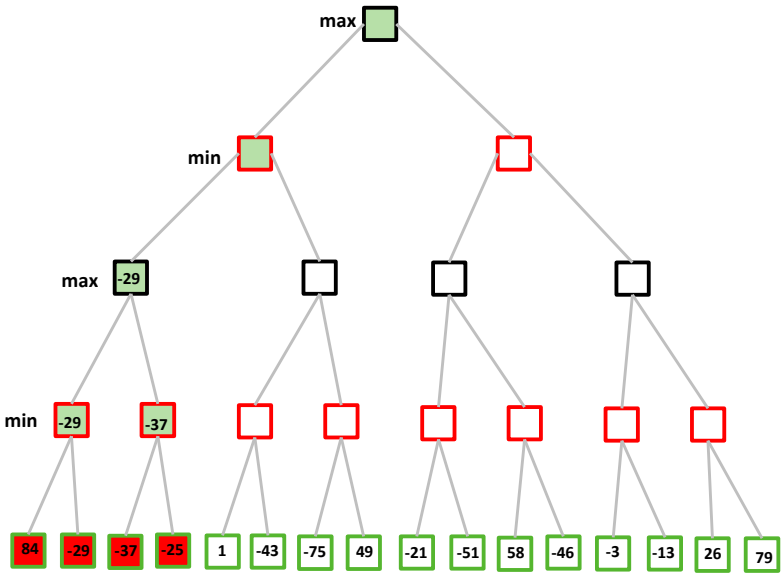
Example





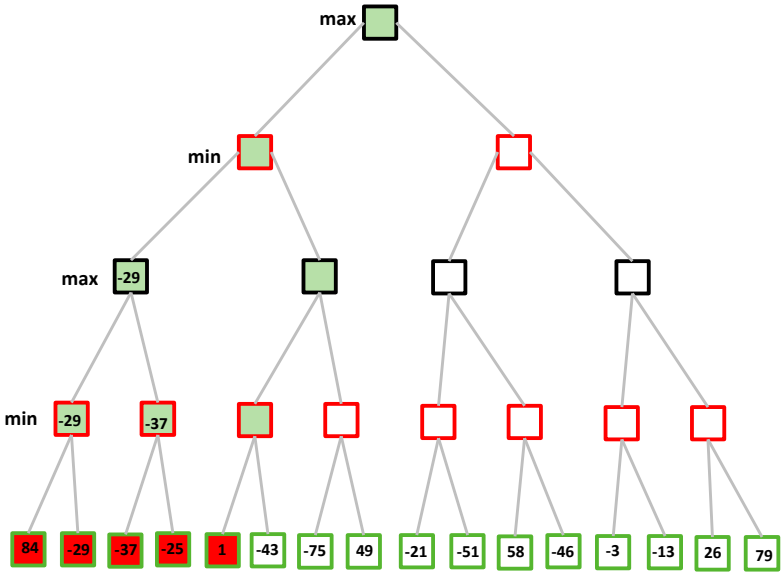
Example





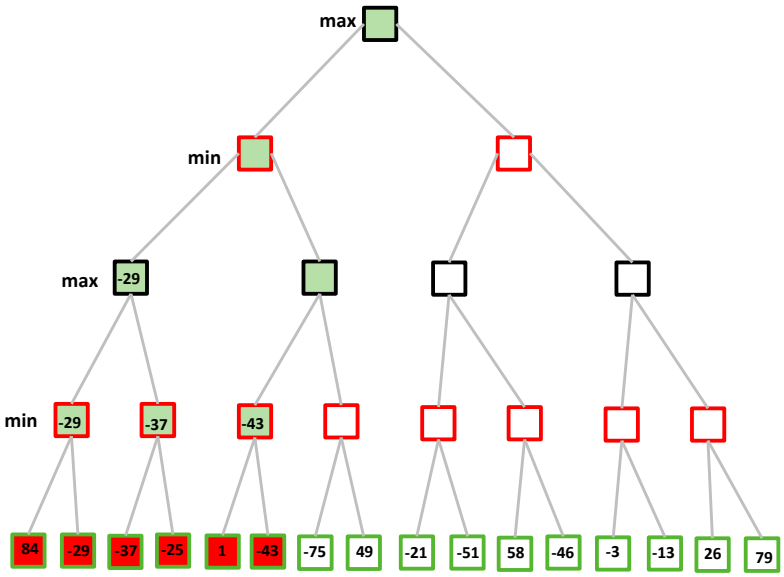
Example





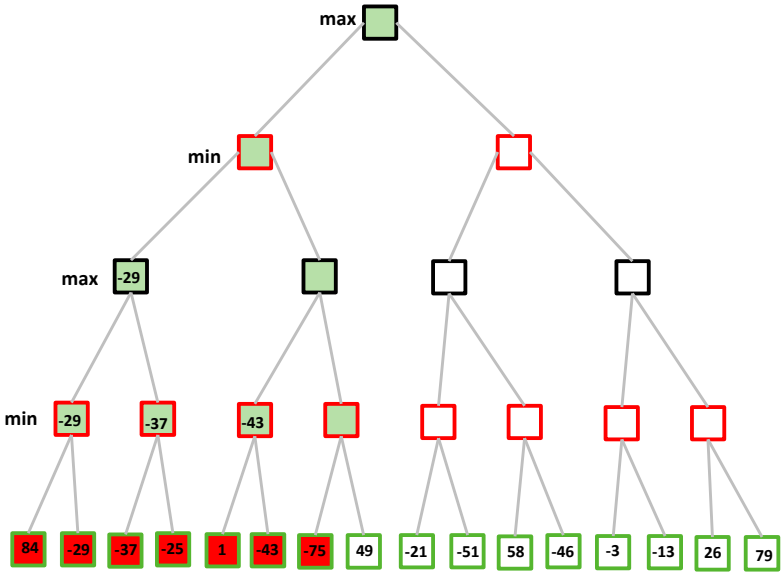
Example



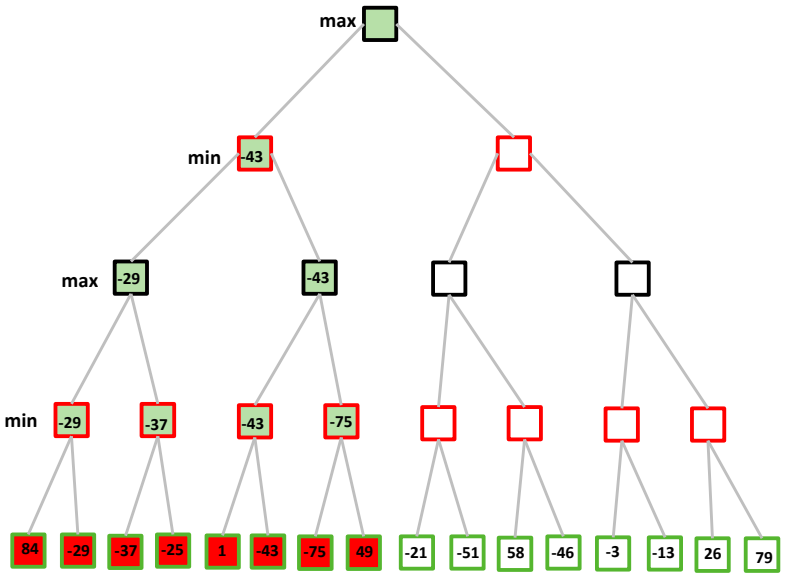


Example



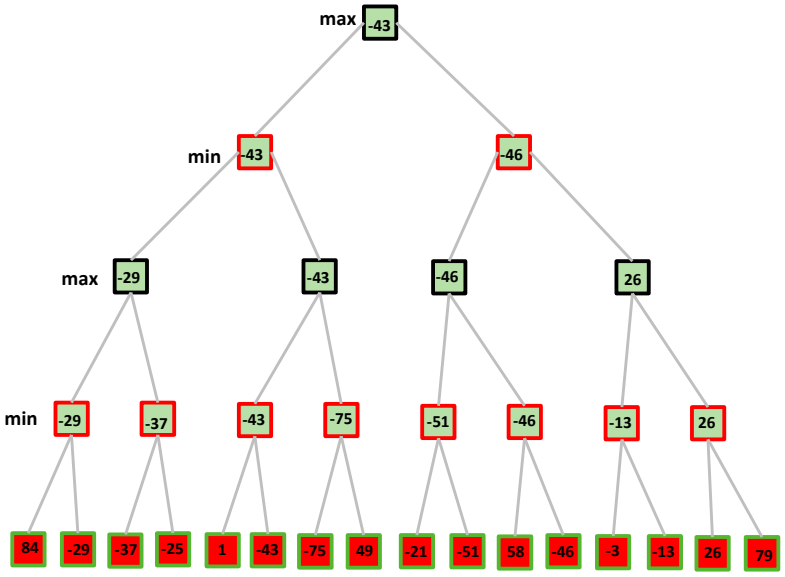


Example

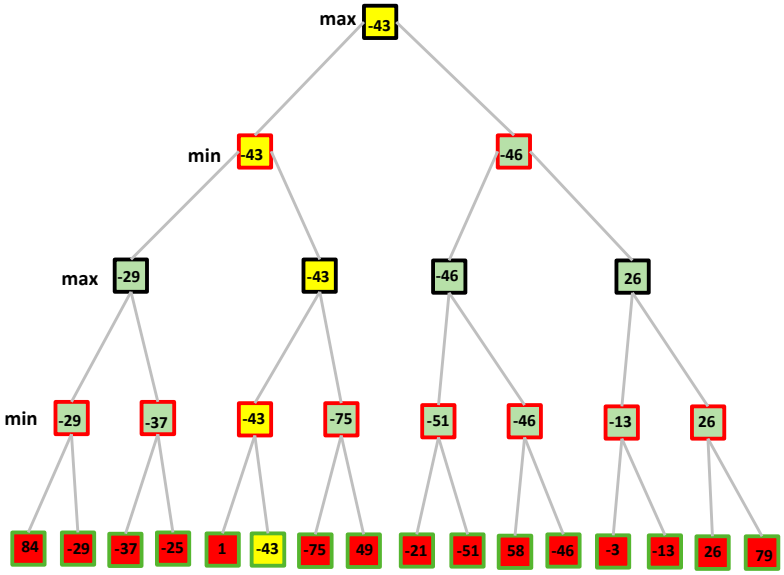


Example

…similarly



Example



Properties of Minimax

- **Complete?**
  - Yes (if tree is finite)
- **Optimal?**
  - Yes (against an optimal opponent)
  - Can it be beaten by an opponent playing sub-optimally?
    - No
- **Time Complexity?**
  - $O(b^m)$
- **Space Complexity?**
  - $O(bm)$ [depth-first search, generate all actions at once]
  - $O(m)$ [backtracking search, generate actions one at a time]

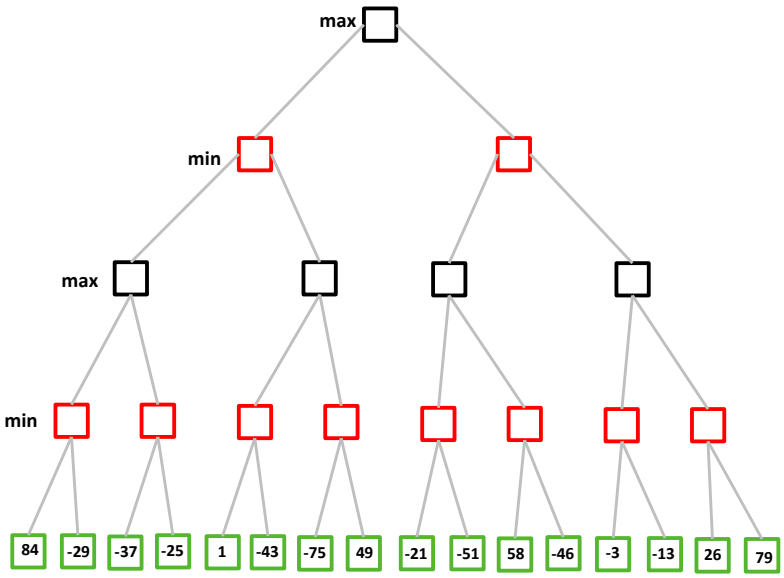
Good Enough?



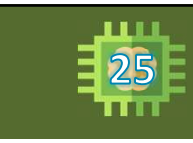
- **Tic-Tac-Toe:**
  - branching factor b≈5 (on average)
  - game length m≈9
  - search space ≈ 5^9≈ 1,953,125
  - Exact solution quite reasonable
- **Chess:**
  - branching factor b≈35
  - game length m≈100
  - search space ≈ 35^{100}≈10^{154}
  - **Exact solution completely infeasible**

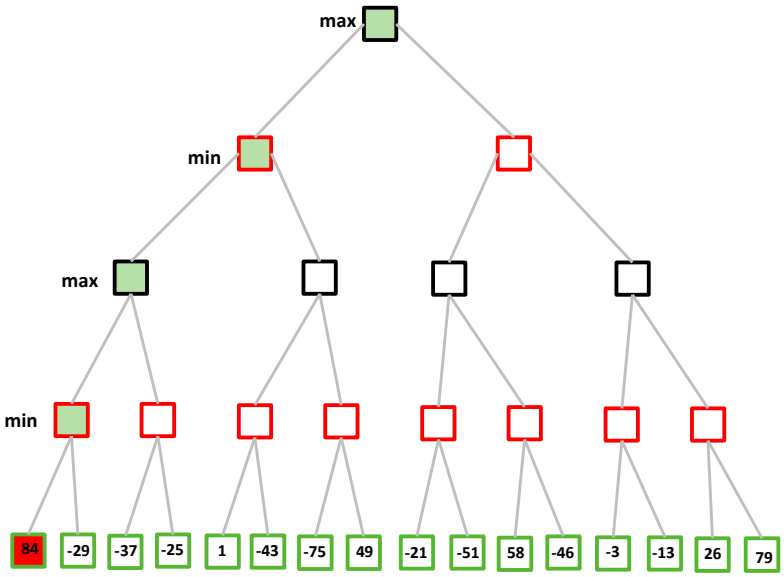
Minimax Example (revisited)





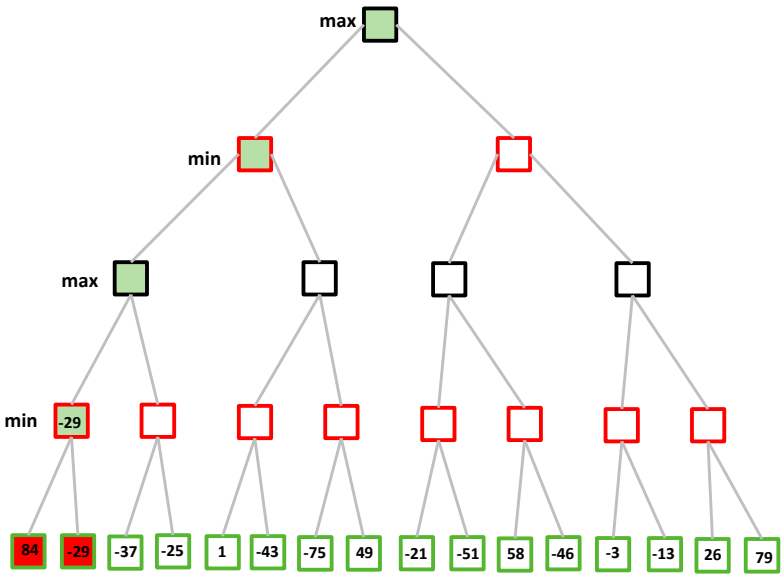
Minimax Example (revisited)



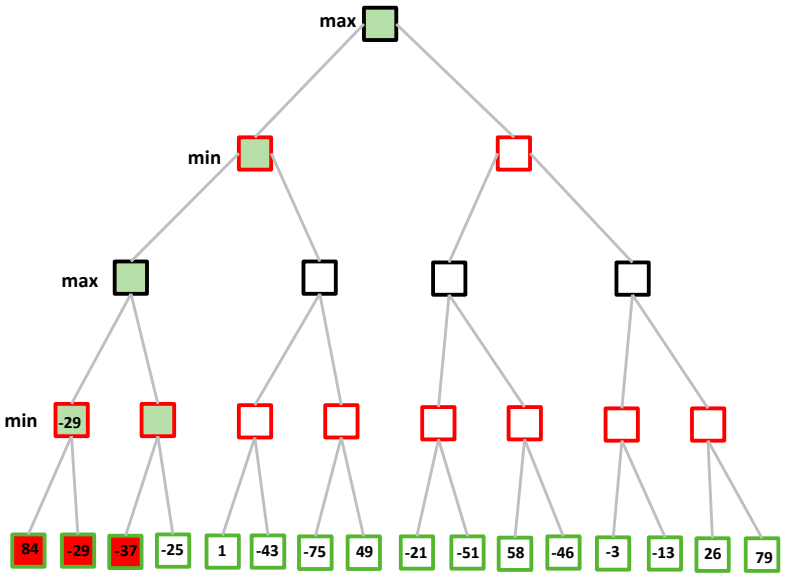


Minimax Example (revisited)

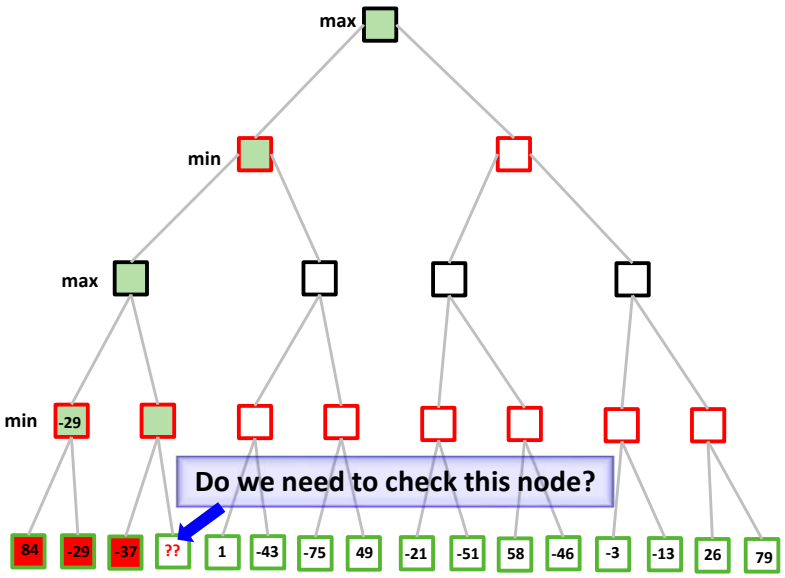




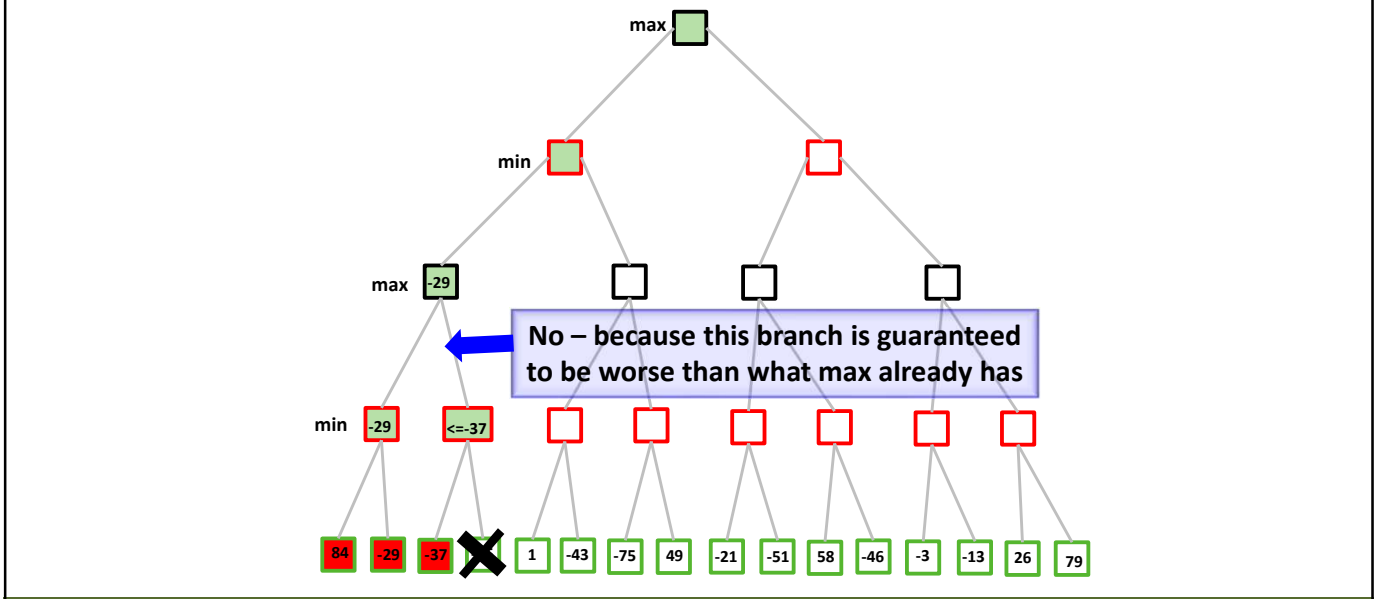
Minimax Example (revisited)



Minimax Example (revisited)



Minimax Example (revisited)



Questions?